

Server-Factory-Additional-Components

Die Nebenbesetzung der Server Factory-Provisioning-Toolchain

Zusammenfassung

Neben Mail Server Factory und dem Core Framework umfasst die Server-Factory-Organisation mehrere kleinere Komponenten: dienstespezifische „Fabriken“ (Web Service, SonarQube, Caching Proxy), deklarative Konfigurationspakete (Docker/Stack/Software-Definitionen) sowie gemeinsame Hilfsprogramme (Utils). Diese konsolidierte Seite beschreibt sie ehrlich – einige sind noch in einem frühen Stadium oder nur rudimentär dokumentiert – und nicht als vollständig spezifizierte Produkte.

Kurzbeschreibung

Eine Gruppe von Server Factory-Unterstützungs-Repositories: Web-Service-Factory, SonarQube-Factory und Caching-Proxy-Factory (dienstespezifische Provisioning-Tools, größtenteils in einem frühen Stadium); Docker/Stack/Software-Definitions (deklarative Konfigurationspakete, die vom Framework genutzt werden) sowie Utils (SSH-Zugriffshilfen und allgemeine Tools). Alle bauen auf dem Core Framework auf.

Ausführliche Beschreibung

Diese Seite fasst die übrigen Server-Factory-Repositories zusammen, da die meisten einzeln betrachtet klein oder bewusst undokumentiert sind und eine Darstellung als fertige Produkte ihren Reifegrad überzeichnen würde. Sie lassen sich in drei Gruppen unterteilen. **Service-Fabriken** folgen dem Mail Server Factory-Muster für andere Serverrollen: Die **Caching-Proxy-Factory** („Betreiben Sie Ihren eigenen Caching-Proxy-Server“) nennt als Hauptmerkmale einen Caching-Proxy, selbstsignierte Zertifikate und einen HTTP-Endpunkt zur Zertifikatsbeschaffung;

die **SonarQube-Factory** („Betreiben Sie Ihren eigenen SonarQube-Server“) richtet sich an die Softwareentwicklung; und die **Web-Service-Factory** instanziiert und konfiguriert einen Webserver zur Bereitstellung von Zielen wie Websites und Microservices. Alle drei sind Kotlin-Projekte, die auf dem Core Framework aufbauen, doch ihre öffentlichen READMEs sind größtenteils Platzhalter („Noch nicht definiert“ für Kompatibilität, Spezifikationen, Einrichtung und Nutzung) – ihre tatsächlichen Fähigkeiten über die genannte Absicht hinaus sind daher UNBESTÄTIGT. **Definitions-Pakete – Docker-Definitions, Stack-Definitions und Software-Definitions** – sind deklarative Konfigurations-Repositories, die das Framework nutzt, um zu wissen, wie Docker-Images, Stacks und Software erstellt/bereitgestellt werden; es handelt sich um versionsgebundene Datenpakete, nicht um Anwendungen. **Utils** stellt allgemeine Hilfsprogramme für die Toolchain bereit, darunter ein Skript `init_ssh_access.sh`, das einen SSH-Schlüssel generiert und auf einem Remote-Host installiert, um für nachfolgende Provisionierungen passwortlosen Root-Zugriff zu ermöglichen. Zusammen ergänzen diese Komponenten die Provisioning-Toolchain rund um das Flaggschiff Mail Server Factory.

Warum wir es entwickelt haben

Das Server Factory-Modell ist darauf ausgelegt, verallgemeinerbar zu sein: Wenn man einen Mailserver aus einer deklarativen Beschreibung provisionieren kann, sollte derselbe Mechanismus auch Webserver, Caching-Proxys und Code-Quality-Server bereitstellen können – gespeist von wiederverwendbaren Definitionspaketen und gemeinsamen Hilfsprogrammen statt von rollenspezifischer Logik. Diese Repositories sind diese Verallgemeinerung in Arbeit und erweitern das bewährte Muster auf neue Servertypen. Ihr Wert liegt hier als Beleg für die Reichweite des Modells; ihr Reifegrad variiert, und diese Seite macht bewusst deutlich, welche Komponenten noch in Entwicklung sind und welche bereits ausgereift.

Warum es ein Game-Changer ist (sachlich betrachtet)

Als Gesamtkonzept zeigen sie, wie das Core Framework über verschiedene Servertypen hinweg wiederverwendbar ist und deklarative Daten (Definitions) von der Ausführung (Factories) trennt. Die einzelnen Service-Factories befinden sich in einer frühen Entwicklungsphase und sollten als Richtungsvorgabe, nicht als fertige Produkte präsentiert werden.

Was innovativ ist

- Ein einheitliches Provisioning-Framework, das für Mail-, Web-, Caching-Proxy- und SonarQube-Rollen generalisiert wurde.
- Deklarative Definitions-Pakete (Docker/Stack/Software), die vom Ausführungsmodul entkoppelt sind.
- Gemeinsame Utils (z. B. ein Ein-Kommando-Passwortlos-Bootstrap für SSH), die in allen Factories wiederverwendet werden.

Herausforderungen & Lösungen

- **Einheitliches Framework für verschiedene Serverrollen:** gelöst durch den Aufbau jeder Factory auf dem Core Framework.
- **Trennung von Konfiguration und Code:** umgesetzt durch die Definitions-Repos als versionsgebundene Datenpakete.
- **(UNGEPRÜFT):** Die READMEs der Service-Factories sind Platzhalter; ihre Implementierungsvollständigkeit lässt sich aus öffentlichen Dokumenten nicht verifizieren – daher als frühe Entwicklungsphase darstellen.

Technologie-Stack (Begründung + Umsetzung)

- **Kotlin** – Web-Service-Factory, SonarQube-Factory, Caching-Proxy-Factory (auf Basis des Core Frameworks).
- **Shell** – Utils und die Definitions-Pakete (Skripte/Konfigurationen).
- **Gradle** – `./gradlew test-Build-` und Testablauf für alle Factories.
- **Docker** – Zielumgebung, beschrieben durch Docker-Definitionen.
- **SSH / OpenSSH** – Passwortloser Zugriffs-Bootstrap der Utils.
- **SonarQube** – Der Server, den die SonarQube-Factory bereitstellt (und gegen den Mail Server Factory eine saubere Prüfung durchführt).

Hinweis zur Transparenz: Die meisten dieser Repos sind Forks innerhalb der Organisation; die Service-Factories sind nur mit Platzhalter-Dokumentation versehen und gemäß Verfassung §11.4.6 als UNGEPRÜFT gekennzeichnet. Sie rangieren klar hinter Mail Server Factory und dem Core Framework.